Case Study Project Report

# Exploratory Data Analysis, Visualization and Deep Learning on the Telco Customer Churn Dataset

Course: Data Exploration and Visualization (CO3 / CO4 / CO5)

| Field | Details |
|---|---|
| Student Name | Kaushikun Krishna Kumar |
| Roll No. | 23AM027 |
| Department | BE CSE(AIML) |
| Year | IV YEAR |
| Date of Submission | 15-10-2026 |
| GitHub Repository | https://github.com/[your-id]/churn-eda-dl |

## 2. Abstract

Customer churn is costly for telecom companies, so identifying the customers likely to leave is a practical priority. This project applies exploratory data analysis, statistical testing and visualization to the IBM Telco Customer Churn dataset, which has 7,043 customers and 21 fields. After fixing a hidden missing-value pattern in TotalCharges and checking duplicates and outliers, twelve visualizations and distribution analyses were produced. The results show that churn (26.5%) is driven by month-to-month contracts, short tenure, fiber-optic service, electronic-check payment and the absence of support services, while gender is irrelevant. A multi-layer perceptron, tuned with a grid search, reached a test ROC-AUC of 0.845 and a recall of 0.70 for churners, slightly below simple baselines such as gradient boosting (0.851).

*Keywords: customer churn, exploratory data analysis, data visualization, multi-layer perceptron, telecom.*

# 3. Introduction and Objective

Telecommunication companies operate in a saturated market where acquiring a new customer costs several times more than retaining an existing one. Customer churn, the event of a subscriber discontinuing service, therefore directly reduces revenue. Understanding which customer attributes are associated with churn, and predicting which customers are at risk, lets a company direct retention offers at the right people.

This project treats churn as a real-world data exploration and visualization problem. The work follows the seven stages of visualization described by Fry [5]: acquire, parse, filter, mine, represent, refine and interact.

## Objectives

- Load and clean a real-world dataset with at least 15 fields and document every data-quality decision.
- Explore individual variables, their probability distributions and the relationships between variables using suitable statistics and visualizations.
- Create at least five meaningful visualizations, each with an explanation of what it shows, why it was made and what was learned.
- Build and tune a deep learning model (multi-layer perceptron) to predict churn, and evaluate it with loss, accuracy, confusion matrix, ROC and precision-recall visualizations.
- Derive practical recommendations and discuss limitations and future work.

## 4. Dataset Description

Source: IBM Telco Customer Churn sample dataset [1], a widely used public dataset describing a fictional telecom company in California. Each row is one customer, and the target field Churn records whether the customer left within the last month. The file is included in the repository (data/Telco-Customer-Churn.csv).

| Property | Value |
|---|---|
| Rows (customers) | 7,043 |
| Columns | 21 (20 predictors + 1 target) |
| Numeric fields | 3 continuous (tenure, MonthlyCharges, TotalCharges) + SeniorCitizen (0/1) |
| Categorical fields | 16 (15 predictors + Churn) |
| Target | Churn (Yes = 1,869 customers, 26.5%; No = 5,174 customers, 73.5%) |
| Missing values | 11 blank TotalCharges entries (all have tenure = 0) |
| Duplicate rows | 0 |

**Table 1: Field dictionary**

| Field | Type | Description |
|---|---|---|
| customerID | Text | Unique customer identifier (dropped) |
| gender | Categorical | Male / Female |
| SeniorCitizen | Binary | Customer is 65 or older |
| Partner | Categorical | Has a partner |
| Dependents | Categorical | Has dependents |
| tenure | Numeric | Months with the company (0-72) |
| PhoneService | Categorical | Has phone service |
| MultipleLines | Categorical | Has multiple phone lines |
| InternetService | Categorical | DSL / Fiber optic / No |
| OnlineSecurity | Categorical | Online security add-on |
| OnlineBackup | Categorical | Online backup add-on |
| DeviceProtection | Categorical | Device protection add-on |
| TechSupport | Categorical | Tech support add-on |
| StreamingTV | Categorical | Streams TV |
| StreamingMovies | Categorical | Streams movies |
| Contract | Categorical | Month-to-month / One year / Two year |
| PaperlessBilling | Categorical | Paperless billing enabled |
| PaymentMethod | Categorical | Electronic check / Mailed check / Bank transfer / Credit card |
| MonthlyCharges | Numeric | Current monthly bill (USD) |
| TotalCharges | Numeric | Total billed to date (USD) |

| Field | Type | Description |
|---|---|---|
| Churn | Target | Customer left (Yes/No) |

**Table 2: Basic statistics of numeric fields**

| Feature | Mean | Std | Min | Median | Max | Skewness |
|---|---|---|---|---|---|---|
| tenure (months) | 32.37 | 24.56 | 0 | 29 | 72 | 0.24 |
| MonthlyCharges (USD) | 64.76 | 30.09 | 18.25 | 70.35 | 118.75 | -0.22 |
| TotalCharges (USD) | 2279.73 | 2266.79 | 0 | 1394.55 | 8684.80 | 0.96 |

# 5. EDA and Preprocessing

## 5.1 Methods used

- **Data type repair.** TotalCharges was stored as text because 11 entries were blank strings. It was converted to numeric.
- **Missing values.** All 11 blank entries belong to customers with tenure = 0 (new customers not yet billed). They were therefore filled with 0, which is the logically correct value, rather than a mean or median that would invent charges.
- **Duplicates.** No duplicated rows or customer IDs were found. customerID was dropped as it has no predictive meaning.
- **Category consolidation.** The labels "No internet service" and "No phone service" in seven fields are equivalent to "No" (the InternetService and PhoneService fields already carry that information), so they were merged to avoid redundant categories.
- **Outliers.** The 1.5 x IQR rule found zero outliers in tenure, MonthlyCharges and TotalCharges, so no rows were removed or capped.
- **Encoding.** Yes/No fields and gender were mapped to 0/1. InternetService, Contract and PaymentMethod were one-hot encoded with the first level dropped, which gives 23 model features.
- **Scaling.** The three numeric features were standardized (z-score). The scaler was fit on the training set only to prevent data leakage.
- **Splitting.** A stratified 70 / 15 / 15 split gave 4,929 training, 1,057 validation and 1,057 test customers, each with a churn rate of about 26.5%.
- **Class imbalance.** Class weights (0.68 for stay, 1.88 for churn) were used during training.

## 5.2 EDA insights

- Churn is imbalanced: 26.5% churners, so accuracy alone is misleading (a model that always predicts "No" scores 73.5%).
- Missing data was not random: it was structurally tied to brand-new customers, so domain reasoning gave a better fix than statistical imputation.
- TotalCharges is almost determined by tenure x MonthlyCharges (correlation with tenure 0.83), so it is partly redundant.
- No feature contains statistical outliers; the numeric variables are bounded business quantities.

# 6. Data Visualization

Twelve visualizations are presented below, covering the seven stages of visualization: Acquire (loading), Parse (types and structure), Filter (cleaning and summaries), Mine (statistical tests and distributions), Represent (the plots), Refine (labelled, colour-consistent figures and the preprocessing needed for modelling) and Interact (an interactive Plotly chart saved as figures/interactive_scatter.html).

## 6.1 Visual 1: Target class distribution

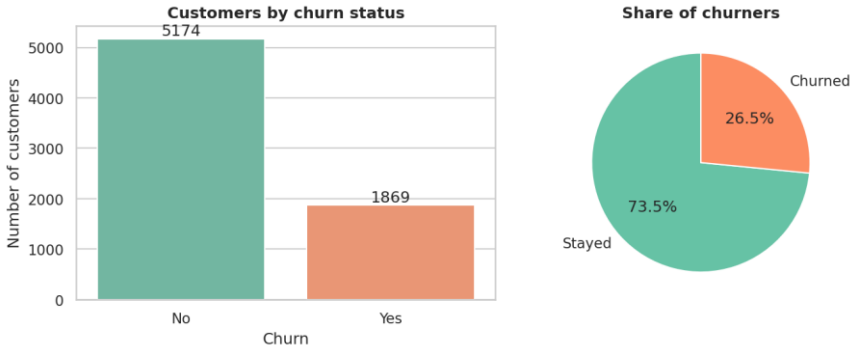



*Figure 1: Number and share of customers who stayed or churned*

- **What it shows:** A bar chart and pie chart of how many customers stayed versus churned.
- **Why it was created:** Class balance determines which evaluation metrics and training strategy are appropriate.
- **Insights:** Only 26.5% of customers churn (about 2.8 : 1 imbalance). Accuracy is misleading, so ROC-AUC, recall and F1 are used and class weights are applied.

## 6.2 Visual 2: Distribution of numerical features

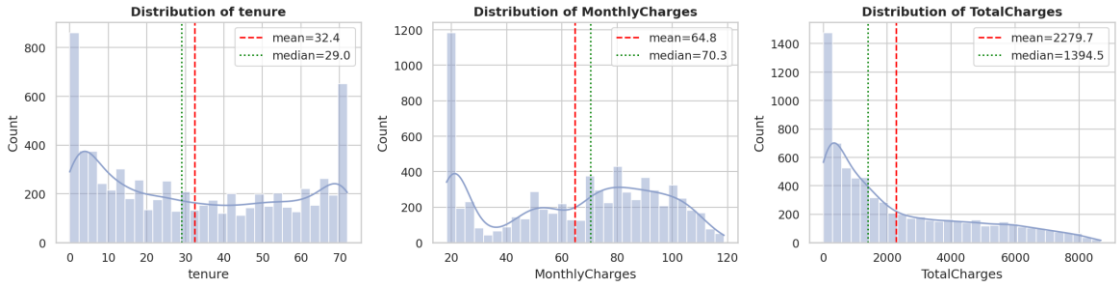



*Figure 2: Histograms with density curves for tenure, MonthlyCharges and TotalCharges*

- **What it shows:** Histograms with KDE, mean and median for each numeric feature.
- **Why it was created:** To see the shape, skew and modality of each variable and decide whether scaling is needed.
- **Insights:** tenure is U-shaped (many new and many loyal customers). MonthlyCharges is multimodal with a cluster near $20 (phone-only plans) and another at $70-$100. TotalCharges is right-skewed (0.96) because it accumulates with tenure.

## 6.3 Visual 3: Boxplots by churn status

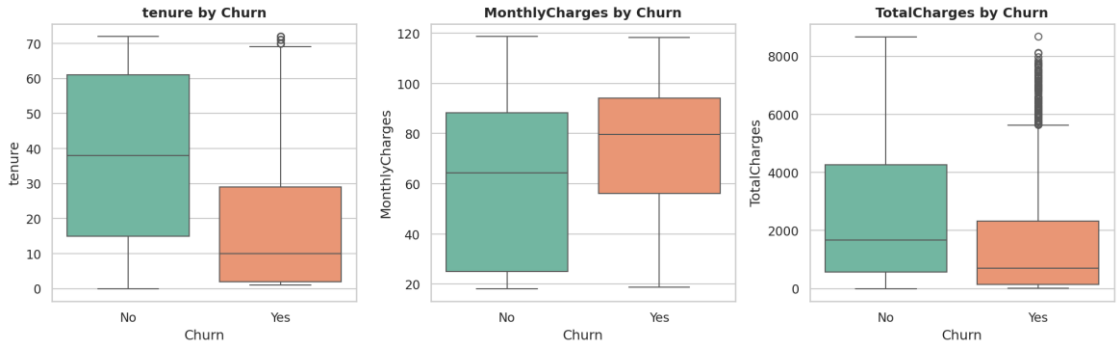



*Figure 3: Boxplots of numeric features split by churn status*

- **What it shows:** Median, spread and range of each numeric feature for churners and non-churners.
- **Why it was created:** To identify which numeric variables separate the two groups.
- **Insights:** Churners have a median tenure of 10 months versus 38 for those who stay, and pay more per month (median $79.6 versus $64.4). Their lower TotalCharges simply reflects shorter tenure.

## 6.4 Visual 4: Churn rate by categorical features

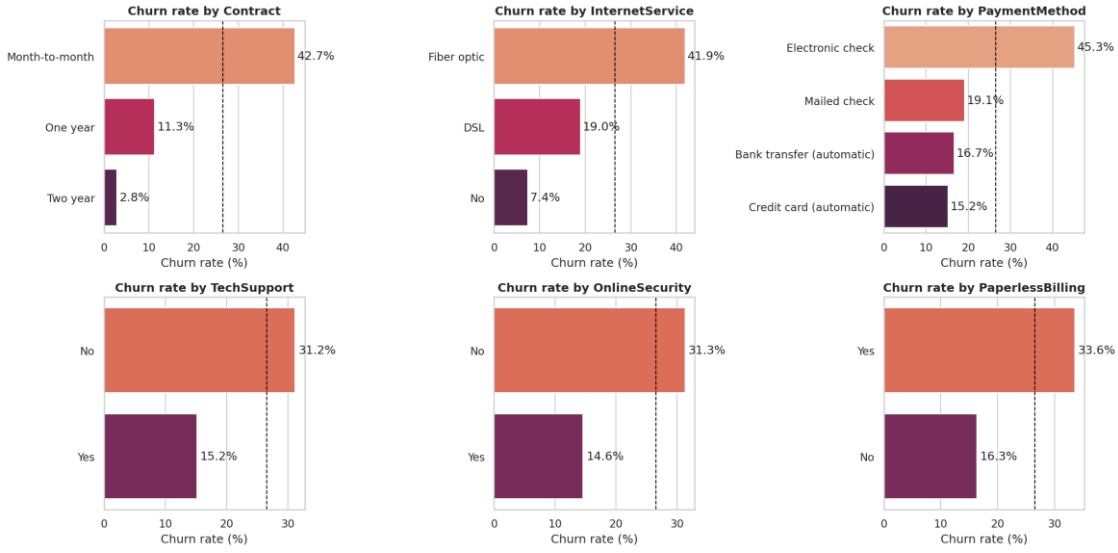



*Figure 4: Churn rate (%) for key categorical features; dashed line is the overall rate*

- **What it shows:** The percentage of customers who churn in each category of six key fields.
- **Why it was created:** Rates are easier to compare across categories than raw counts, and the categorical fields dominate this dataset.
- **Insights:** Month-to-month contracts churn at 42.7% versus 2.8% for two-year contracts. Fiber-optic customers churn at 41.9% versus 19.0% for DSL. Electronic-check payers churn most (45.3%). Customers without tech support or online security churn at roughly twice the rate of those who have them (about 31% versus 15%).

## 6.5 Visual 5: Correlation heatmap

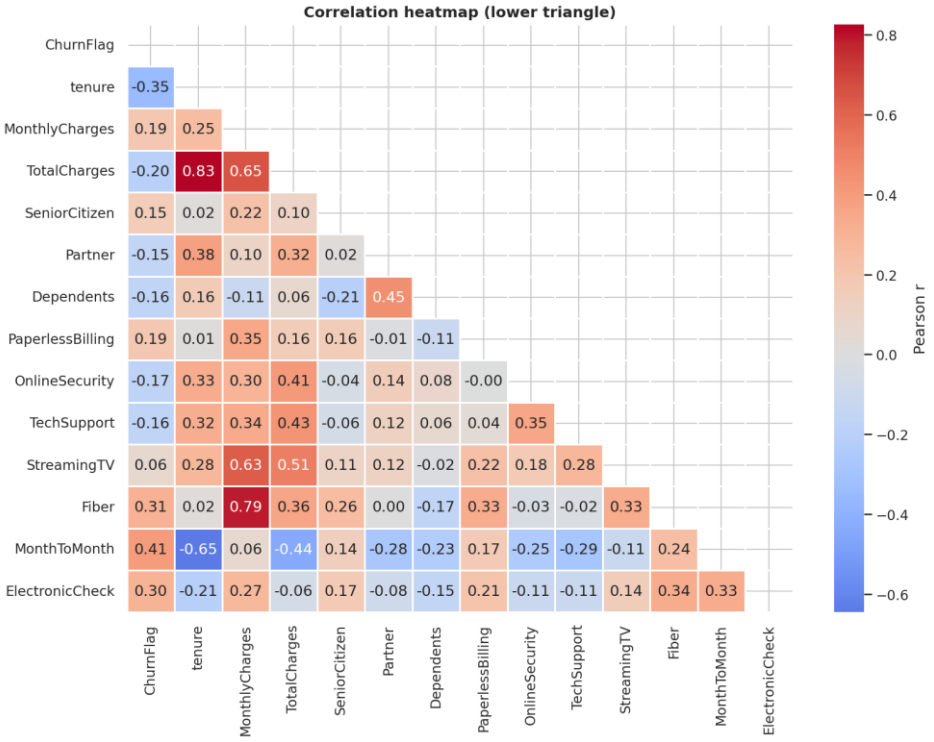



*Figure 5: Correlation heatmap of numeric and encoded binary features*

- **What it shows:** Pairwise Pearson correlations (lower triangle).
- **Why it was created:** To find the strongest linear relationships with churn and detect multicollinearity.
- **Insights:** Churn correlates negatively with tenure (r = -0.35) and positively with month-to-month contracts (+0.41), fiber optic (+0.31) and electronic check (+0.30). tenure and TotalCharges are strongly correlated (0.83), confirming redundancy.

## 6.6 Visual 6: Scatter plot of tenure versus monthly charges

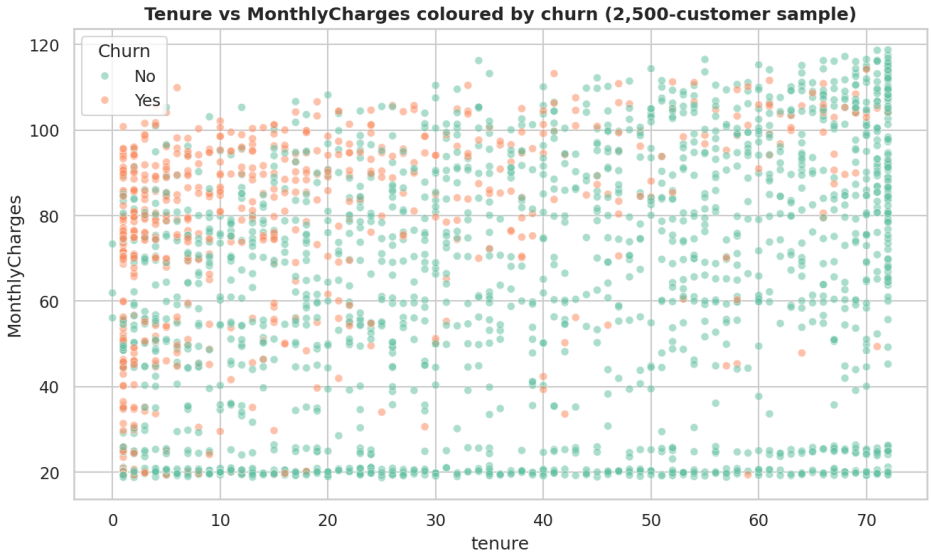



*Figure 6: Tenure versus MonthlyCharges coloured by churn (2,500-customer sample)*

- **What it shows:** Each point is a customer, coloured by churn status.
- **Why it was created:** To view two numeric drivers jointly and look for clusters.
- **Insights:** Churners concentrate in the top-left region (short tenure and high charges). Customers who stay beyond about 40 months rarely leave regardless of price.

## 6.7 Visual 7: Pair plot

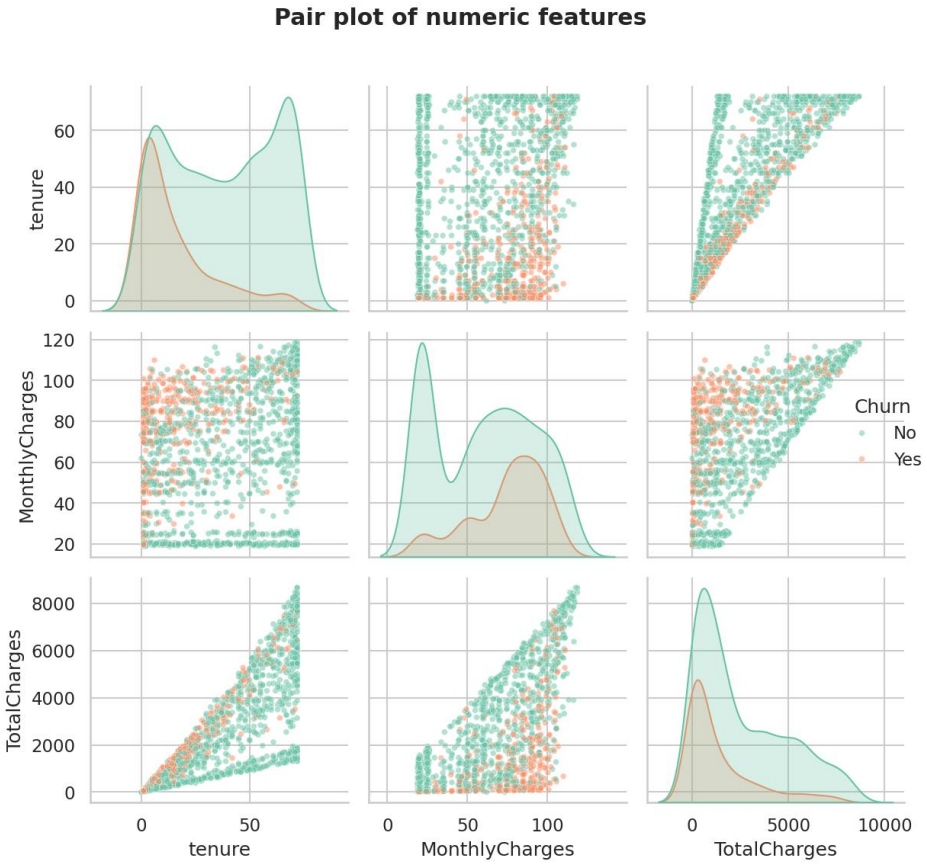



*Figure 7: Pair plot of the three numeric features by churn status*

- **What it shows:** All pairwise scatter plots plus class-wise density curves.
- **Why it was created:** A one-shot multivariate check of class separability.
- **Insights:** The churn class has a density peak at low tenure. The tenure and TotalCharges relationship fans out from the origin, confirming redundancy. No single pair separates the classes perfectly, which justifies a non-linear model.

## 6.8 Visual 8: Heatmap of churn rate by contract and tenure group

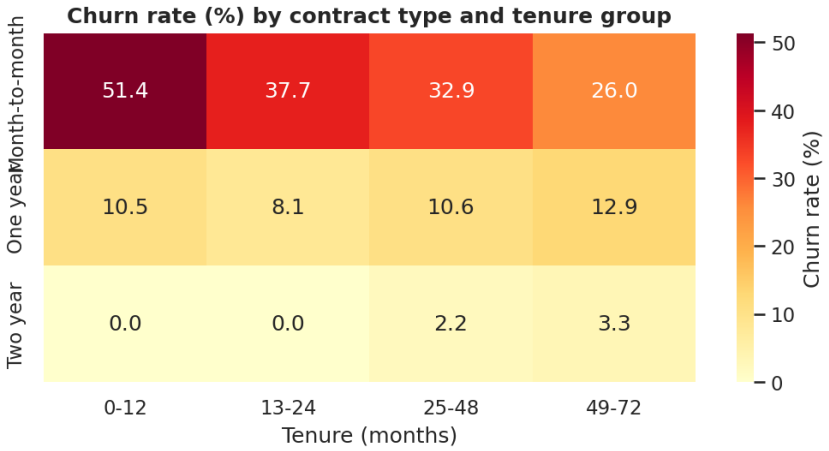



*Figure 8: Churn rate (%) by contract type and tenure group*

- **What it shows:** Churn rate in every contract-by-tenure cell.
- **Why it was created:** A two-factor view of the two strongest drivers.
- **Insights:** Month-to-month customers in their first year churn at 51.4%. Two-year contracts stay at or below 3.3% at every tenure. The main retention lever is moving new customers off month-to-month contracts.

## 6.9 Visuals 9 and 10: Probability distributions (CO4)

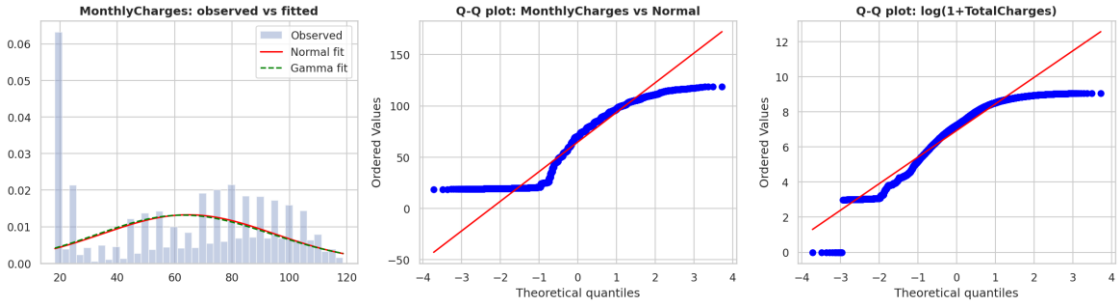



*Figure 9: Fitted distributions and Q-Q plots for MonthlyCharges and TotalCharges*

- **What it shows:** Normal and gamma fits to MonthlyCharges, and Q-Q plots against the normal distribution.
- **Why it was created:** To test whether the numeric variables follow standard continuous distributions.
- **Insights:** None of the three numeric features is normal (D'Agostino K-squared $p < 0.001$). Kolmogorov-Smirnov statistics are about 0.126 for the best-fitting candidate on MonthlyCharges, so no single parametric curve describes its multimodal shape. A log transform over-corrects TotalCharges (skew changes from +0.96 to -0.82).

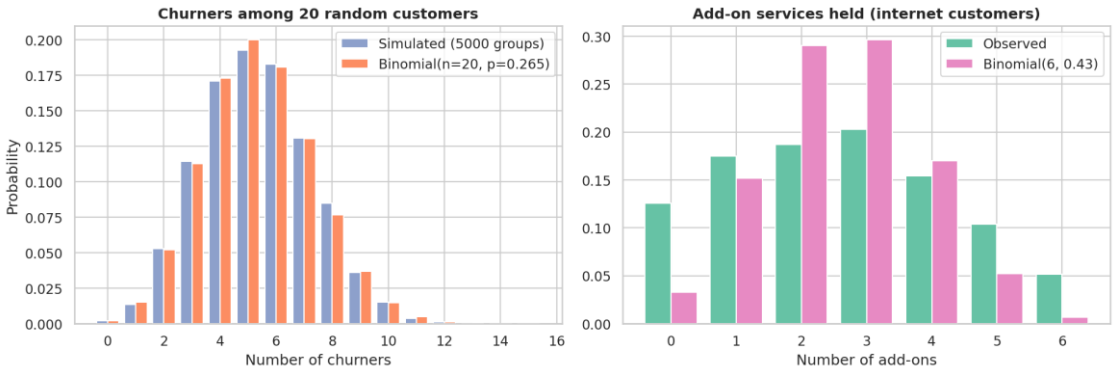



*Figure 10: Binomial model for churners per group of 20 customers and for add-on counts*

- **What it shows:** Left: the number of churners in random groups of 20 customers against a Binomial(20, 0.265). Right: the number of add-on services held by internet customers against a fitted binomial.
- **Why it was created:** To apply discrete probability distributions to the data and check their assumptions.
- **Insights:** The churner count matches the binomial closely (expected 5.31, SD 1.97; P(at least 8 churners in 20) = 0.134), so churn behaves like independent Bernoulli trials at the population level. The add-on count does not: 12.6% of internet customers hold none and 5.1% hold all six, versus about 3% and under 1% expected under the binomial. This over-dispersion shows add-on purchases are not independent.

## 6.10 Visual 11: Association strength with churn

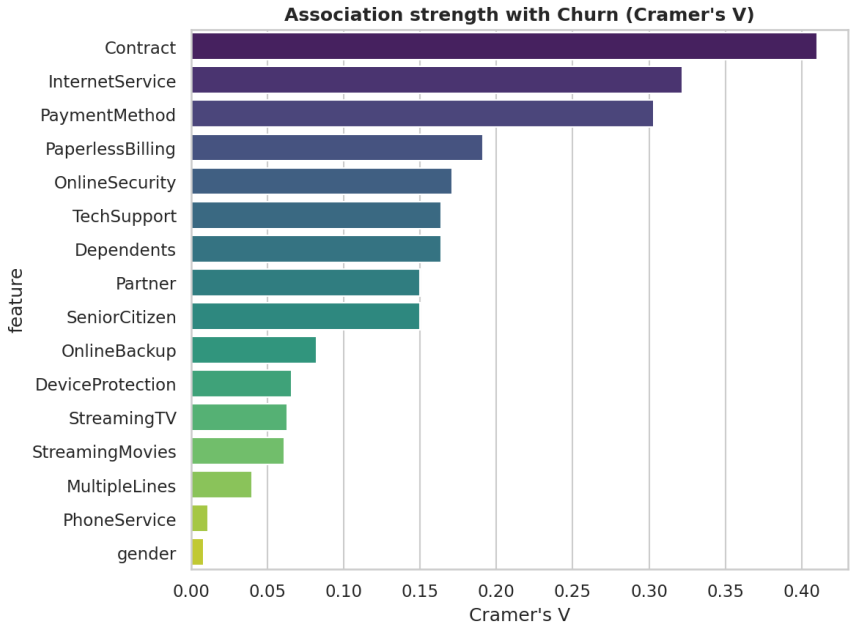



*Figure 11: Cramer's V between each categorical feature and churn*

- **What it shows:** Cramer's V effect sizes from chi-square tests of independence.
- **Why it was created:** Hypothesis tests alone give tiny p-values with 7,000 rows, so an effect-size measure is needed to rank the features.
- **Insights:** Contract (V = 0.41), InternetService (0.32) and PaymentMethod (0.30) are strongest. gender (p = 0.49) and PhoneService (p = 0.34) are statistically unrelated to churn. Mann-

Whitney tests show that all three numeric features differ between the classes, with tenure showing the largest effect (rank-biserial r = -0.48).

## 6.11 Mutual information and interactive chart

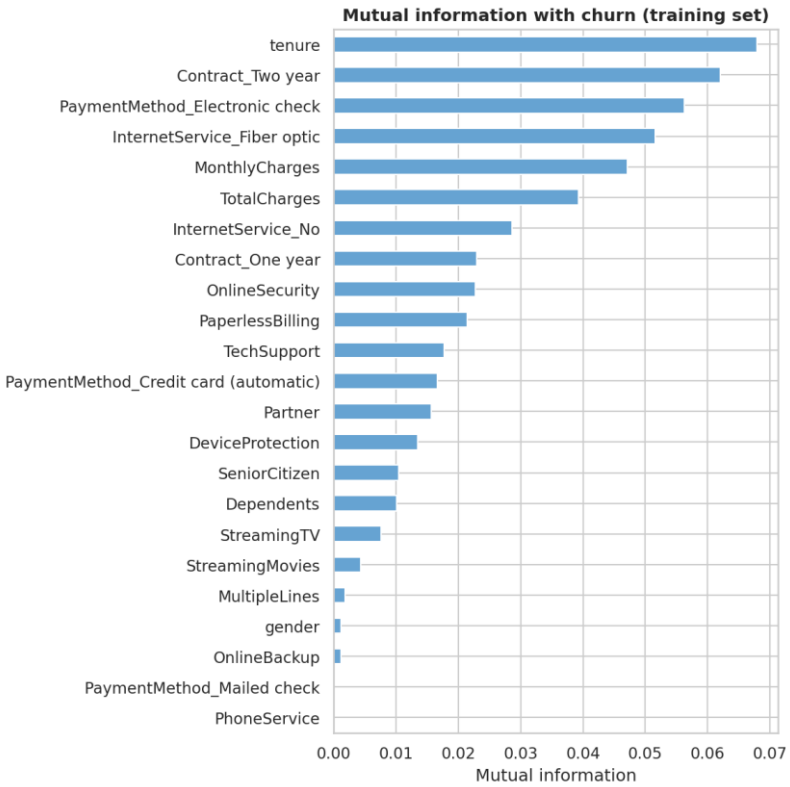



*Figure 12: Mutual information between each model feature and churn (training set)*

Mutual information, which captures non-linear dependence, ranks tenure, Contract_Two year, PaymentMethod_Electronic check, InternetService_Fiber optic and MonthlyCharges highest, which agrees with the statistical tests. The interactive Plotly scatter plot (figures/interactive_scatter.html) completes the Interact stage by letting the reader hover, zoom and filter by contract type.

# 7. Deep Learning Model

## 7.1 Architecture

A multi-layer perceptron (MLP) was selected because the data are tabular with mixed binary, one-hot and numeric inputs and non-linear interactions. It was built with TensorFlow/Keras [4].

| Layer | Output size | Details |
|---|---|---|
| Input | 23 | 23 encoded and scaled features |
| Dense + BatchNorm + ReLU + Dropout | 128 | L2 = 1e-4, dropout 0.4 |
| Dense + BatchNorm + ReLU + Dropout | 64 | L2 = 1e-4, dropout 0.4 |
| Dense + BatchNorm + ReLU + Dropout | 32 | L2 = 1e-4, dropout 0.4 |
| Dense (output) | 1 | Sigmoid: probability of churn |

## 7.2 Training parameters

| Parameter | Value |
|---|---|
| Loss | Binary cross-entropy with class weights (0.68 / 1.88) |
| Optimizer | Adam [7] |
| Learning rate | 0.003 (reduced by half after 4 epochs without validation-loss improvement) |
| Batch size | 128 |
| Maximum epochs | 100 (early stopping on validation AUC, patience 10; stopped after 18 epochs) |
| Regularization | Dropout [6], L2 weight decay, batch normalization |
| Decision threshold | 0.63, chosen on the validation set to maximize F1 |
| Random seed | 42 |

## 7.3 Hyperparameter tuning (basic grid search)

Twelve configurations were compared on validation AUC: hidden layers {(32), (64, 32), (128, 64, 32)} x dropout {0.2, 0.4} x learning rate {0.001, 0.003}. The test set was not used during tuning.

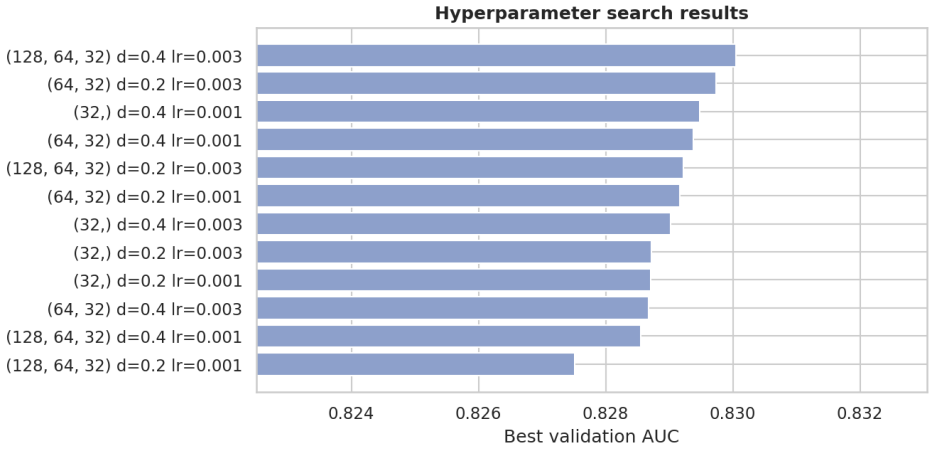



*Figure 13: Best validation AUC for each hyperparameter configuration*

The best configuration was (128, 64, 32) hidden units, dropout 0.4 and learning rate 0.003 (validation AUC 0.830). All twelve configurations fall between 0.8275 and 0.8301, so the architecture matters very little for this dataset and differences of this size are within run-to-run noise.

## 7.4 Feature analysis

Before modelling, feature relevance was analysed with Cramer's V, Mann-Whitney tests and mutual information (Section 6). After training, permutation importance was computed by shuffling each feature on the validation set 10 times and measuring the fall in AUC (Section 8).

# 8. Result Visualization and Interpretation

## 8.1 Loss and accuracy versus epoch

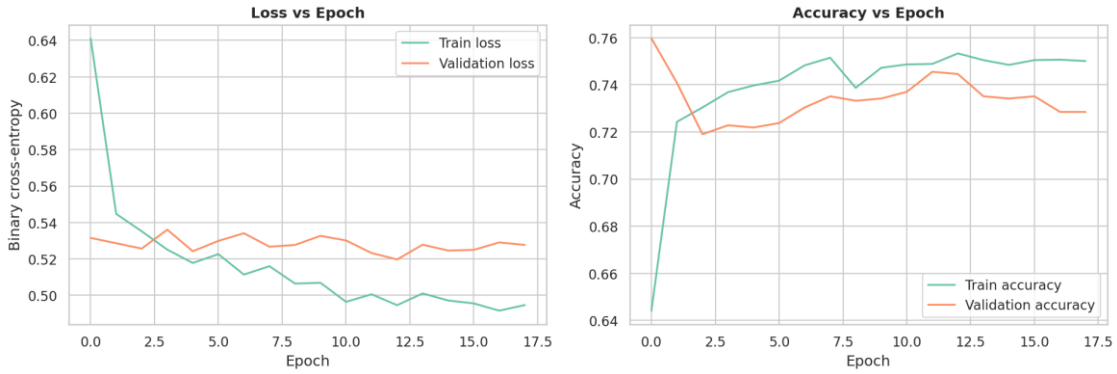



*Figure 14: Training and validation loss and accuracy per epoch*

Training loss keeps falling, while validation loss flattens at about 0.52 to 0.53 after the first few epochs. The widening gap shows mild over-fitting, which is why dropout, L2 regularization, learning-rate reduction and early stopping were used. Accuracy is measured at a 0.5 threshold on a class-weighted model, so it can sit below the 73.5% majority-class figure; this is expected because the model deliberately trades accuracy for recall on the minority class.

## 8.2 Confusion matrix, ROC curve and precision-recall curve

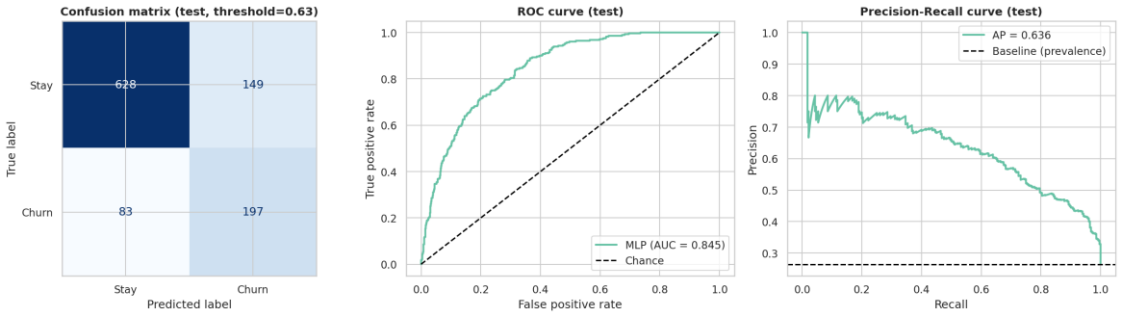



*Figure 15: Test-set confusion matrix, ROC curve and precision-recall curve*

| Test-set metric (1,057 customers) | Value |
|---|---|
| Confusion matrix (TN / FP / FN / TP) | 628 / 149 / 83 / 197 |
| Accuracy | 0.781 |
| Precision (churn) | 0.569 |
| Recall (churn) | 0.704 |
| F1 (churn) | 0.629 |
| ROC-AUC | 0.845 |

The model catches 197 of the 280 churners (70.4% recall) and falsely flags 149 loyal customers. The ROC curve lies well above the chance diagonal with an AUC of 0.845, which means a randomly chosen churner receives a higher score than a randomly chosen non-churner 84.5% of the time. The precision-recall curve stays far above the 26.5% prevalence baseline over a wide range of recall, which is the more honest view for an imbalanced problem.

## 8.3 Score distribution and calibration

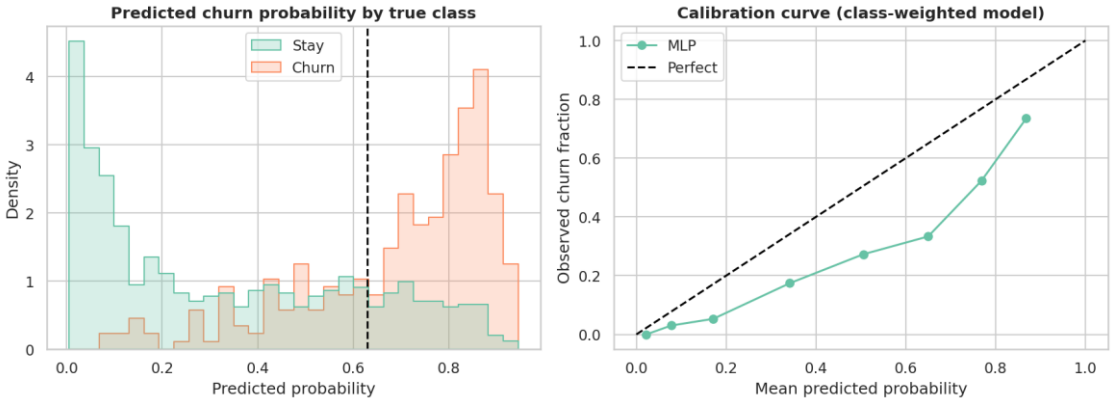



*Figure 16: Predicted probability distribution by class and calibration curve*

The two class score distributions are clearly separated but overlap in the middle, which represents the difficult customers. Because the model was trained with class weights, its probabilities are systematically too high (the calibration curve lies below the diagonal). The ranking is good, but if real probabilities are required, a calibration step such as isotonic regression should be added.

## 8.4 Comparison with classical models




*Figure 17: F1 and ROC-AUC of the MLP and three classical baselines on the test set*

| Model | Accuracy | Precision | Recall | F1 | ROC-AUC |
|---|---|---|---|---|---|
| Logistic Regression | 0.770 | 0.549 | 0.746 | 0.632 | 0.849 |
| Random Forest | 0.761 | 0.532 | 0.804 | 0.640 | 0.850 |
| Gradient Boosting | 0.773 | 0.554 | 0.732 | 0.631 | 0.851 |
| MLP (ours) | 0.781 | 0.569 | 0.704 | 0.629 | 0.845 |

The MLP is within about 0.005 AUC of every baseline. It has the highest accuracy and precision but the lowest AUC. This is typical for small tabular datasets (about 5,000 training rows) and indicates that the signal is mostly additive. The deep model therefore adds no measurable gain over simple models here, which is itself a useful finding.

## 8.5 Permutation feature importance

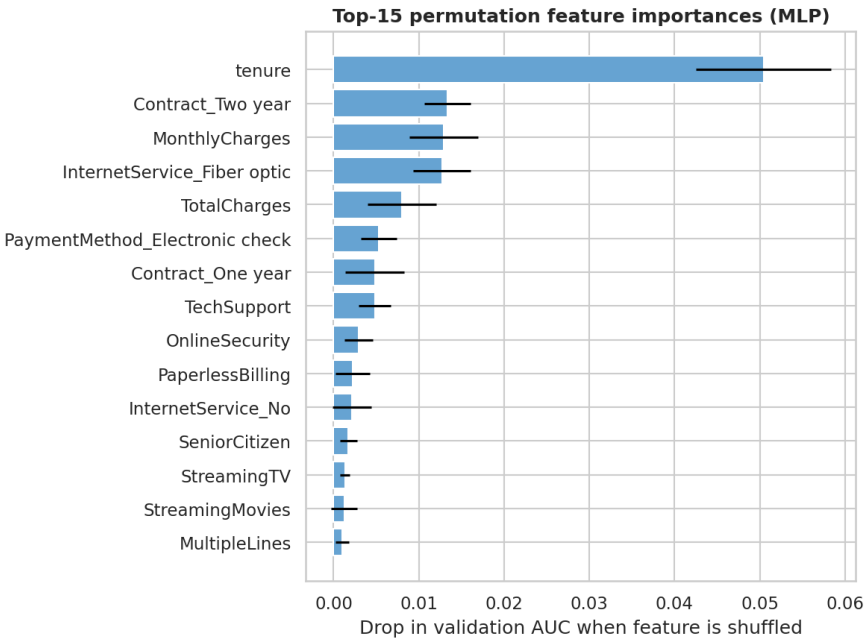



*Figure 18: Top 15 permutation importances of the MLP (drop in validation AUC)*

Shuffling tenure reduces validation AUC by about 0.050, far more than any other feature. Contract_Two year, MonthlyCharges, fiber-optic internet and TotalCharges follow (about 0.008 to 0.013 each). This agrees with the EDA, the statistical tests and the mutual-information ranking, so the findings are robust across methods.

# 9. Conclusion and Future Scope

## 9.1 What was learned

- Churn is 26.5% and is mainly driven by contract flexibility, short tenure, higher monthly price, fiber-optic service, electronic-check payment and lack of tech-support or security add-ons. Gender and phone service have no relationship with churn.
- The first year is critical: month-to-month customers in their first 12 months churn at 51.4%, while two-year contract customers almost never leave (at most 3.3%).
- Numeric features are non-normal and not captured by single parametric distributions, but the overall count of churners in a group follows a binomial distribution.
- The tuned MLP achieves test ROC-AUC 0.845, recall 0.704 and F1 0.629, competitive with but not better than classical baselines.

## 9.2 Practical recommendations

- Offer discounts or incentives to move new month-to-month customers to one- or two-year contracts during the first year.
- Bundle tech support and online security with fiber-optic plans, as customers without them churn about twice as often.
- Encourage a switch from electronic check to automatic payment methods.
- Use the model score to rank customers for retention calls, choosing the threshold according to the cost of a retention offer versus the value of a retained customer.

## 9.3 Challenges

- Class imbalance (about 2.8 : 1), handled with class weights and threshold tuning.
- Hidden missing values stored as blank strings, which made TotalCharges text.
- Redundant, correlated features such as tenure and TotalCharges.
- Non-normal and multimodal numeric distributions.
- A modest dataset size, which limits what deep learning can add, and mild over-fitting.

## 9.4 Limitations

The dataset is a single snapshot of a fictional company without dates, so seasonality and changes over time cannot be studied, and results may not transfer to other operators. The analysis shows association, not causation: for example, lower churn with tech support does not prove that adding tech support would retain customers. Metrics come from one random split and one training seed.

## 9.5 Future scope

- Collect usage, billing-history and customer-service-call data over time and use sequence models (LSTM or Transformer).
- Try SMOTE, focal loss or cost-sensitive learning for imbalance.
- Add SHAP explanations for customer-level reasons.
- Apply probability calibration (isotonic regression or Platt scaling).
- Use repeated k-fold cross-validation and statistical tests of model differences.
- Run A/B tests of retention offers and build a deployed scoring API.

# 10. References


[1] IBM, "Telco customer churn" sample dataset, IBM Telco Customer Churn on IBM Cloud Pak for Data. [Online]. Available: https://github.com/IBM/telco-customer-churn-on-icp4d

[2] I. Goodfellow, Y. Bengio and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[4] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," arXiv:1603.04467, 2015.

[5] B. Fry, Visualizing Data: Exploring and Explaining Data with the Processing Environment. Sebastopol, CA, USA: O'Reilly Media, 2007.

[6] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," J. Mach. Learn. Res., vol. 15, no. 56, pp. 1929-1958, 2014.

[7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2015. arXiv:1412.6980.

[8] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. Int. Conf. Mach. Learn. (ICML), 2015, pp. 448-456.

[9] J. D. Hunter, "Matplotlib: A 2D graphics environment," Comput. Sci. Eng., vol. 9, no. 3, pp. 90-95, 2007.

# 11. Appendix: Code

The complete notebook is in notebooks/churn_eda_dl.ipynb and the same code as a script is in src/churn_eda_dl.py in the GitHub repository. The code cells are reproduced below in order.

**Cell 1**

```
# ---------- Imports and global configuration ----------
import os, warnings, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats


warnings.filterwarnings("ignore")
SEED = 42
os.environ["TF CPP MIN LOG LEVEL"] = "3"
random.seed(SEED); np.random.seed(SEED)


sns.set theme(style="whitegrid", context="notebook", palette="Set2")
plt.rcParams.update({"figure.dpi": 110, "axes.titleweight": "bold"})


FIG DIR = "../figures"; os.makedirs(FIG DIR, exist ok=True)
def savefig(name):
    plt.tight layout(); plt.savefig(f"{FIG DIR}/{name}.png", dpi=150, bbox inches="tight");
plt.show()
```

**Cell 2**

```
DATA_PATH = "../data/Telco-Customer-Churn.csv"
df = pd.read csv(DATA PATH)
print("Shape (rows, columns):", df.shape)
df.head()
```

**Cell 3**

```
df.info()
```

**Cell 4**

```
# Field groups used throughout the notebook
target = "Churn"
numeric cols = ["tenure", "MonthlyCharges", "TotalCharges"]   # TotalCharges is stored as text,
fixed below
binary_like  = ["SeniorCitizen"]
cat_cols = [c for c in df.columns if c not in numeric_cols + [target, "customerID",
"SeniorCitizen"]]
print(len(df.columns), "fields in total")
print("Categorical:", cat_cols)
```

**Cell 5**

```
# --- 1. Data type problem: TotalCharges is text because 11 rows hold a blank string ---
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"].replace(" ", np.nan), errors="coerce")


# --- 2. Missing values ---
miss = df.isna().sum(); print("Missing values per column:\n", miss[miss > 0])
print("\nTenure of rows with missing TotalCharges:", df.loc[df.TotalCharges.isna(),
"tenure"].unique())
```

**Cell 6**

```
# All 11 blanks belong to brand-new customers (tenure = 0) who have not been billed yet.
# So the correct value is 0 (not the mean/median), which avoids inventing charges.
df["TotalCharges"] = df["TotalCharges"].fillna(0.0)


# --- 3. Duplicates ---
print("Duplicate rows       :", df.duplicated().sum())
print("Duplicate customerIDs:", df["customerID"].duplicated().sum())


# --- 4. Drop identifier (carries no predictive information) ---
df = df.drop(columns="customerID")


# --- 5. Consistency: 'No internet service' / 'No phone service' are equivalent to 'No' ---
for c in
["MultipleLines","OnlineSecurity","OnlineBackup","DeviceProtection","TechSupport","StreamingTV","Str
eamingMovies"]:
    df[c] = df[c].replace({"No internet service": "No", "No phone service": "No"})


# --- 6. Target encoding for analysis ---
df["ChurnFlag"] = (df["Churn"] == "Yes").astype(int)
print("Missing values after cleaning:", int(df.isna().sum().sum()))
```

**Cell 7**

```
# --- 7. Outlier detection with the IQR rule (1.5 x IQR) ---
rows = []
for c in numeric cols:
    q1, q3 = df[c].quantile([.25, .75]); iqr = q3 - q1
    lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
    rows.append([c, lo, hi, ((df[c] < lo) | (df[c] > hi)).sum()])
pd.DataFrame(rows, columns=["feature", "lower_fence", "upper_fence", "n_outliers"])
```

**Cell 8**

```
df[numeric_cols + ["SeniorCitizen"]].describe().T.round(2)
```

**Cell 9**

```
# Skewness and kurtosis (shape of each numeric distribution)
shape = pd.DataFrame({"skew": df[numeric cols].skew(), "kurtosis":
df[numeric_cols].kurt()}).round(3)
shape
```

**Cell 10**

```
# Frequency tables for categorical features
for c in ["Contract", "InternetService", "PaymentMethod"]:
    print(df[c].value counts(normalize=True).mul(100).round(1).to string(), "\n")
print("Churn rate overall: %.2f%%" % (df.ChurnFlag.mean()*100))
```

**Cell 11**

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
counts = df["Churn"].value_counts()
sns.countplot(data=df, x="Churn", order=["No","Yes"], ax=ax[0], palette=["#66c2a5","#fc8d62"])
for p in ax[0].patches:
    ax[0].annotate(f"{int(p.get height())}", (p.get x()+p.get width()/2, p.get height()),
ha="center", va="bottom")
ax[0].set title("Customers by churn status"); ax[0].set ylabel("Number of customers")
ax[1].pie(counts[["No","Yes"]], labels=["Stayed","Churned"], autopct="%1.1f%%",
colors=["#66c2a5","#fc8d62"], startangle=90)
ax[1].set_title("Share of churners")
savefig("v1_churn_distribution")
```

**Cell 12**

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, c in zip(axes, numeric cols):
    sns.histplot(df[c], kde=True, bins=30, ax=ax, color="#8da0cb")
    ax.axvline(df[c].mean(), color="red", ls="--", label=f"mean={df[c].mean():.1f}")
    ax.axvline(df[c].median(), color="green", ls=":", label=f"median={df[c].median():.1f}")
    ax.set title(f"Distribution of {c}"); ax.legend()
savefig("v2_numeric_histograms")
```

**Cell 13**

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4.5))
for ax, c in zip(axes, numeric_cols):
    sns.boxplot(data=df, x="Churn", y=c, order=["No","Yes"], ax=ax, palette=["#66c2a5","#fc8d62"])
    ax.set_title(f"{c} by Churn")
savefig("v3 boxplots by churn")
print(df.groupby("Churn")[numeric_cols].median().round(1))
```

**Cell 14**

```
fig, axes = plt.subplots(2, 3, figsize=(16, 8))
for ax, c in zip(axes.ravel(),
["Contract","InternetService","PaymentMethod","TechSupport","OnlineSecurity","PaperlessBilling"]):
    rate = df.groupby(c)["ChurnFlag"].mean().mul(100).sort values(ascending=False)
    sns.barplot(x=rate.values, y=rate.index, ax=ax, palette="rocket r")
    ax.axvline(df.ChurnFlag.mean()*100, color="k", ls="--", lw=1)
    for i, v in enumerate(rate.values): ax.text(v+0.5, i, f"{v:.1f}%", va="center")
    ax.set_title(f"Churn rate by {c}"); ax.set_xlabel("Churn rate (%)"); ax.set_ylabel("")
savefig("v4_churn_rate_by_category")
```

**Cell 15**

```
# Numeric view of the data (binary Yes/No mapped to 1/0) so that all fields can be correlated
corr_df = df.copy()
for c in
["Partner","Dependents","PhoneService","PaperlessBilling","MultipleLines","OnlineSecurity","OnlineBa
ckup",
         "DeviceProtection","TechSupport","StreamingTV","StreamingMovies"]:
    corr df[c] = (corr df[c] == "Yes").astype(int)
corr df["gender"] = (corr df["gender"] == "Male").astype(int)
corr_df["Fiber"] = (corr_df["InternetService"] == "Fiber optic").astype(int)
corr df["MonthToMonth"] = (corr df["Contract"] == "Month-to-month").astype(int)
corr_df["ElectronicCheck"] = (corr_df["PaymentMethod"] == "Electronic check").astype(int)
corr cols =
["ChurnFlag","tenure","MonthlyCharges","TotalCharges","SeniorCitizen","Partner","Dependents","Paperl
essBilling",
            "OnlineSecurity","TechSupport","StreamingTV","Fiber","MonthToMonth","ElectronicCheck"]
corr = corr df[corr cols].corr()
plt.figure(figsize=(11, 8.5))
```

```
sns.heatmap(corr, mask=np.triu(np.ones like(corr, dtype=bool)), annot=True, fmt=".2f",
cmap="coolwarm", center=0, linewidths=.5, cbar kws={"label":"Pearson r"})
plt.title("Correlation heatmap (lower triangle)")
savefig("v5_correlation_heatmap")
print(corr["ChurnFlag"].drop("ChurnFlag").sort_values().round(3).to_string())
```

**Cell 16**

```
plt.figure(figsize=(9, 5.5))
sns.scatterplot(data=df.sample(2500, random_state=SEED), x="tenure", y="MonthlyCharges",
hue="Churn", hue order=["No","Yes"],
                palette=["#66c2a5","#fc8d62"], alpha=.55, s=28)
plt.title("Tenure vs MonthlyCharges coloured by churn (2,500-customer sample)")
savefig("v6_scatter_tenure_monthly")
```

**Cell 17**

```
g = sns.pairplot(df.sample(1500, random state=SEED), vars=numeric cols, hue="Churn",
hue_order=["No","Yes"],
                 palette=["#66c2a5","#fc8d62"], plot_kws={"alpha":.5, "s":14}, diag_kind="kde")
g.fig.suptitle("Pair plot of numeric features", y=1.02, fontweight="bold")
savefig("v7_pairplot")
```

**Cell 18**

```
df["TenureGroup"] = pd.cut(df["tenure"], bins=[-1,12,24,48,72], labels=["0-12","13-24","25-48","49-
72"])
pivot = df.pivot_table(index="Contract", columns="TenureGroup", values="ChurnFlag",
aggfunc="mean").mul(100)
plt.figure(figsize=(7, 3.8))
sns.heatmap(pivot, annot=True, fmt=".1f", cmap="YlOrRd", cbar kws={"label":"Churn rate (%)"})
plt.title("Churn rate (%) by contract type and tenure group"); plt.ylabel(""); plt.xlabel("Tenure
(months)")
savefig("v8_heatmap_contract_tenure")
```

**Cell 19**

```
# ---- Continuous: fit candidate distributions to MonthlyCharges and tenure; compare with
Kolmogorov-Smirnov ----
cands = {"norm": stats.norm, "lognorm": stats.lognorm, "gamma": stats.gamma, "expon": stats.expon,
"weibull_min": stats.weibull_min}
res = []
for col in ["MonthlyCharges", "tenure"]:
    x = df[col][df[col] > 0].values
    for name, dist in cands.items():
        params = dist.fit(x)
        ks = stats.kstest(x, name, args=params)
        res.append([col, name, round(ks.statistic, 4), ks.pvalue])
fit tbl = pd.DataFrame(res, columns=["feature", "distribution", "KS stat", "p value"])
fit_tbl.sort_values(["feature","KS_stat"])
```

**Cell 20**

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
# Panel A: MonthlyCharges with normal vs gamma fits
x = df["MonthlyCharges"].values; grid = np.linspace(x.min(), x.max(), 300)
axes[0].hist(x, bins=40, density=True, alpha=.5, color="#8da0cb", label="Observed")
axes[0].plot(grid, stats.norm.pdf(grid, *stats.norm.fit(x)), "r-", label="Normal fit")
axes[0].plot(grid, stats.gamma.pdf(grid, *stats.gamma.fit(x)), "g--", label="Gamma fit")
axes[0].set title("MonthlyCharges: observed vs fitted"); axes[0].legend()
# Panel B: Q-Q plot against normal
stats.probplot(x, dist="norm", plot=axes[1]); axes[1].set title("Q-Q plot: MonthlyCharges vs
Normal")
# Panel C: Q-Q plot for TotalCharges after log transform
stats.probplot(np.log1p(df["TotalCharges"]), dist="norm", plot=axes[2]); axes[2].set_title("Q-Q
plot: log(1+TotalCharges)")
savefig("v9_distribution_fits_qq")


# Normality tests (D'Agostino K^2 is reliable for large n; Shapiro on a 5000-sample)
for c in numeric_cols:
    k2 = stats.normaltest(df[c]); print(f"{c:15s} D'Agostino K2 p = {k2.pvalue:.2e} -> {'NOT normal'
if k2.pvalue<0.05 else 'normal'}")
```

**Cell 21**

```
# ---- Discrete 1: Bernoulli / Binomial model for churn ----
p = df.ChurnFlag.mean(); n = 20
k = np.arange(0, 16)
pmf = stats.binom.pmf(k, n, p)
# Empirical: draw many random groups of 20 customers and count churners in each group
rng = np.random.default_rng(SEED)
sim = np.array([df.ChurnFlag.sample(n, random state=int(s)).sum() for s in rng.integers(0, 10**6,
5000)])
emp = np.bincount(sim, minlength=16)[:16] / len(sim)


# ---- Discrete 2: number of add-on services per customer (0..6) ----
addons =
["OnlineSecurity","OnlineBackup","DeviceProtection","TechSupport","StreamingTV","StreamingMovies"]
df["n addons"] = (df[addons] == "Yes").sum(axis=1)
inet = df[df.InternetService != "No"]["n_addons"]       # only internet customers can buy add-ons
```

```
pb = inet.mean() / 6
obs = inet.value_counts(normalize=True).sort_index().reindex(range(7), fill_value=0)


fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
axes[0].bar(k-0.2, emp, width=.4, label="Simulated (5000 groups)", color="#8da0cb")
axes[0].bar(k+0.2, pmf, width=.4, label=f"Binomial(n=20, p={p:.3f})", color="#fc8d62")
axes[0].set title("Churners among 20 random customers"); axes[0].set xlabel("Number of churners");
axes[0].set ylabel("Probability"); axes[0].legend()
axes[1].bar(np.arange(7)-0.2, obs.values, width=.4, label="Observed", color="#66c2a5")
axes[1].bar(np.arange(7)+0.2, stats.binom.pmf(np.arange(7), 6, pb), width=.4, label=f"Binomial(6,
{pb:.2f})", color="#e78ac3")
axes[1].set title("Add-on services held (internet customers)"); axes[1].set xlabel("Number of add-
ons"); axes[1].legend()
savefig("v10 discrete distributions")
print("Binomial: E[churners in 20] =", round(n*p,2), " SD =", round(np.sqrt(n*p*(1-p)),2))
print("P(at least 8 churners among 20) = %.4f" % stats.binom.sf(7, n, p))
```

**Cell 22**

```
# --- Numeric vs churn: Mann-Whitney U (non-parametric, since features are not normal) ---
rows = []
for c in numeric_cols:
    a, b = df.loc[df.ChurnFlag==0, c], df.loc[df.ChurnFlag==1, c]
    u = stats.mannwhitneyu(a, b); rbc = 1 - 2*u.statistic/(len(a)*len(b))   # rank-biserial effect
size
    rows.append([c, u.pvalue, round(rbc, 3)])
print(pd.DataFrame(rows,
columns=["feature","MWU_p_value","rank_biserial_effect"]).to_string(index=False))


# --- Categorical vs churn: chi-square test of independence + Cramer's V effect size ---
def cramers_v(ct):
    chi2 = stats.chi2 contingency(ct)[0]; n = ct.values.sum()
    return np.sqrt(chi2 / (n * (min(ct.shape) - 1)))
rows = []
for c in cat_cols + ["SeniorCitizen"]:
    ct = pd.crosstab(df[c], df["Churn"]); chi2, p , dof,   = stats.chi2 contingency(ct)
    rows.append([c, round(chi2,1), p , round(cramers v(ct),3)])
chi tbl = pd.DataFrame(rows,
columns=["feature","chi2","p value","cramers V"]).sort values("cramers V", ascending=False)
chi_tbl.round(4)
```

**Cell 23**

```
plt.figure(figsize=(8, 6))
sns.barplot(data=chi tbl, x="cramers V", y="feature", palette="viridis")
plt.title("Association strength with Churn (Cramer's V)"); plt.xlabel("Cramer's V")
savefig("v11_cramers_v")
```

**Cell 24**

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler


model df = df.drop(columns=["Churn", "TenureGroup", "n addons"]).copy()
y = model_df.pop("ChurnFlag").values


# Binary Yes/No and gender -> 0/1 ; multi-class -> one-hot (drop first avoids the dummy variable
trap)
bin cols =
["Partner","Dependents","PhoneService","PaperlessBilling","MultipleLines","OnlineSecurity","OnlineBa
ckup",
           "DeviceProtection","TechSupport","StreamingTV","StreamingMovies"]
for c in bin cols: model df[c] = (model df[c] == "Yes").astype(int)
model df["gender"] = (model df["gender"] == "Male").astype(int)
X = pd.get_dummies(model_df, columns=["InternetService","Contract","PaymentMethod"],
drop first=True).astype(float)
print("Feature matrix:", X.shape); print(list(X.columns))


# 70 / 15 / 15 stratified split
X tmp, X test, y tmp, y test = train test split(X, y, test size=0.15, stratify=y, random state=SEED)
X train, X val, y train, y val = train test split(X tmp, y tmp, test size=0.15/0.85, stratify=y tmp,
random_state=SEED)


# Standardize numeric columns (fit on train only)
scaler = StandardScaler().fit(X train[numeric cols])
for part in (X_train, X_val, X_test):
    part[numeric_cols] = scaler.transform(part[numeric_cols])


print(f"Train {X_train.shape[0]} | Val {X_val.shape[0]} | Test {X_test.shape[0]}")
print("Churn rate  train/val/test:", [round(v.mean(),3) for v in (y_train, y_val, y_test)])
```

**Cell 25**

```
from sklearn.feature selection import mutual info classif
mi = pd.Series(mutual_info_classif(X_train, y_train, random_state=SEED),
index=X_train.columns).sort_values()
```

```
plt.figure(figsize=(8, 8)); mi.plot.barh(color="#66a3d2")
plt.title("Mutual information with churn (training set)"); plt.xlabel("Mutual information")
savefig("v12 mutual information")
print(mi.sort_values(ascending=False).head(8).round(4).to_string())
```

**Cell 26**

```
import plotly.express as px
fig = px.scatter(df.sample(2000, random state=SEED), x="tenure", y="MonthlyCharges", color="Churn",
symbol="Contract",
                 hover data=["InternetService", "PaymentMethod", "TotalCharges"],
                 color discrete map={"No":"#66c2a5","Yes":"#fc8d62"}, title="Interactive: tenure vs
monthly charges")
fig.write html(f"{FIG DIR}/interactive scatter.html")
fig.show() if False else print("Interactive chart saved to figures/interactive_scatter.html (open in
a browser)")
```

**Cell 27**

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
tf.keras.utils.set_random_seed(SEED)


# Class weights compensate for the 73.5 / 26.5 imbalance
cw = {0: len(y_train) / (2*np.sum(y_train==0)), 1: len(y_train) / (2*np.sum(y_train==1))}
print("Class weights:", {k: round(v,3) for k,v in cw.items()})


def build_mlp(input_dim, units=(64, 32), dropout=0.3, lr=1e-3, l2=1e-4):
    # Fully-connected network: [Dense -> BatchNorm -> ReLU -> Dropout] x N  ->  Dense(1, sigmoid)
    inp = keras.Input(shape=(input_dim,))
    x = inp
    for u in units:
        x = layers.Dense(u, kernel regularizer=keras.regularizers.l2(l2))(x)
        x = layers.BatchNormalization()(x)
        x = layers.Activation("relu")(x)
        x = layers.Dropout(dropout)(x)
    out = layers.Dense(1, activation="sigmoid")(x)
    m = keras.Model(inp, out)
    m.compile(optimizer=keras.optimizers.Adam(lr), loss="binary_crossentropy",
              metrics=["accuracy", keras.metrics.AUC(name="auc")])
    return m

build_mlp(X_train.shape[1]).summary()
```

**Cell 28**

```
import itertools, time
grid = {"units": [(32,), (64, 32), (128, 64, 32)], "dropout": [0.2, 0.4], "lr": [1e-3, 3e-3]}
early = lambda: keras.callbacks.EarlyStopping(monitor="val auc", mode="max", patience=8,
restore_best_weights=True)
results = []; t0 = time.time()
for units, dr, lr in itertools.product(grid["units"], grid["dropout"], grid["lr"]):
    tf.keras.utils.set random seed(SEED)
    m = build_mlp(X_train.shape[1], units, dr, lr)
    h = m.fit(X train.values, y train, validation data=(X val.values, y val), epochs=60,
batch size=128,
              class weight=cw, callbacks=[early()], verbose=0)
    results.append({"units": str(units), "dropout": dr, "lr": lr, "best val auc":
max(h.history["val_auc"]),
                    "epochs run": len(h.history["loss"])})
tune = pd.DataFrame(results).sort_values("best_val_auc", ascending=False).reset_index(drop=True)
print(f"Grid search finished in {time.time()-t0:.0f}s")
tune.round(4)
```

**Cell 29**

```
best = tune.iloc[0]; print("Best configuration:\n", best)
import ast
BEST UNITS = ast.literal eval(best["units"]); BEST DROPOUT = float(best["dropout"]); BEST LR =
float(best["lr"])


plt.figure(figsize=(9, 4.5))
lab = tune.apply(lambda r: f"{r.units} d={r.dropout} lr={r.lr}", axis=1)
plt.barh(lab[::-1], tune["best val auc"][::-1], color="#8da0cb"); plt.xlim(tune.best val auc.min()-
0.005, tune.best_val_auc.max()+0.003)
plt.xlabel("Best validation AUC"); plt.title("Hyperparameter search results")
savefig("v13_hyperparameter_search")
```

**Cell 30**

```
tf.keras.utils.set_random_seed(SEED)
model = build mlp(X train.shape[1], BEST UNITS, BEST DROPOUT, BEST LR)
callbacks = [keras.callbacks.EarlyStopping(monitor="val auc", mode="max", patience=10,
restore best weights=True),
             keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=4,
min_lr=1e-5)]
```

```
history = model.fit(X train.values, y train, validation data=(X val.values, y val), epochs=100,
batch size=128,
                    class weight=cw, callbacks=callbacks, verbose=0)
print("Epochs trained:", len(history.history["loss"]))
```

**Cell 31**

```
hist = pd.DataFrame(history.history)
fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
ax[0].plot(hist["loss"], label="Train loss"); ax[0].plot(hist["val_loss"], label="Validation loss")
ax[0].set_title("Loss vs Epoch"); ax[0].set_xlabel("Epoch"); ax[0].set_ylabel("Binary cross-
entropy"); ax[0].legend()
ax[1].plot(hist["accuracy"], label="Train accuracy"); ax[1].plot(hist["val_accuracy"],
label="Validation accuracy")
ax[1].set_title("Accuracy vs Epoch"); ax[1].set_xlabel("Epoch"); ax[1].set_ylabel("Accuracy");
ax[1].legend()
savefig("r1_loss_accuracy_curves")
```

**Cell 32**

```
from sklearn.metrics import (confusion_matrix, roc_curve, roc_auc_score, classification_report,
ConfusionMatrixDisplay,
                             accuracy score, precision score, recall score, f1 score,
precision recall curve, average precision score)
p test = model.predict(X test.values, verbose=0).ravel()
p_val  = model.predict(X_val.values, verbose=0).ravel()


# Choose the decision threshold on the VALIDATION set (maximise F1), then apply once to the test set
ths = np.linspace(0.2, 0.8, 61)
best_th = ths[np.argmax([f1_score(y_val, p_val >= t) for t in ths])]
print(f"Threshold chosen on validation set: {best_th:.2f}")
pred test = (p test >= best th).astype(int)


fig, ax = plt.subplots(1, 3, figsize=(17, 4.8))
ConfusionMatrixDisplay(confusion_matrix(y_test, pred_test),
display labels=["Stay","Churn"]).plot(ax=ax[0], cmap="Blues", colorbar=False)
ax[0].set_title(f"Confusion matrix (test, threshold={best_th:.2f})")
fpr, tpr,   = roc curve(y test, p test); auc = roc auc score(y test, p test)
ax[1].plot(fpr, tpr, lw=2, label=f"MLP (AUC = {auc:.3f})"); ax[1].plot([0,1],[0,1],"k--",
label="Chance")
ax[1].set xlabel("False positive rate"); ax[1].set ylabel("True positive rate");
ax[1].set_title("ROC curve (test)"); ax[1].legend()
pr, rc,   = precision recall curve(y test, p test); ap = average precision score(y test, p test)
ax[2].plot(rc, pr, lw=2, label=f"AP = {ap:.3f}"); ax[2].axhline(y_test.mean(), color="k", ls="--",
label="Baseline (prevalence)")
ax[2].set_xlabel("Recall"); ax[2].set_ylabel("Precision"); ax[2].set_title("Precision-Recall curve
(test)"); ax[2].legend()
savefig("r2 confusion roc pr")
print(classification_report(y_test, pred_test, target_names=["Stay","Churn"], digits=3))
```

**Cell 33**

```
from sklearn.calibration import calibration_curve
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
sns.histplot(x=p_test, hue=np.where(y_test==1,"Churn","Stay"), bins=30, stat="density",
common norm=False, element="step", ax=ax[0], palette={"Churn":"#fc8d62","Stay":"#66c2a5"},
hue_order=["Stay","Churn"])
ax[0].axvline(best th, color="k", ls="--"); ax[0].set title("Predicted churn probability by true
class"); ax[0].set xlabel("Predicted probability")
fp, mp = calibration curve(y test, p test, n bins=8, strategy="quantile")
ax[1].plot(mp, fp, "o-", label="MLP"); ax[1].plot([0,1],[0,1],"k--", label="Perfect");
ax[1].set_xlabel("Mean predicted probability")
ax[1].set ylabel("Observed churn fraction"); ax[1].set title("Calibration curve (class-weighted
model)"); ax[1].legend()
savefig("r3_score_distribution_calibration")
```

**Cell 34**

```
from sklearn.linear model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier


models = {"Logistic Regression": LogisticRegression(max iter=2000, class weight="balanced"),
          "Random Forest": RandomForestClassifier(300, min_samples_leaf=5, class_weight="balanced",
random state=SEED, n jobs=-1),
          "Gradient Boosting": GradientBoostingClassifier(random_state=SEED)}
rows = []
for name, m in models.items():
    m.fit(X_train, y_train); pr_ = m.predict_proba(X_test)[:, 1]
    th = ths[np.argmax([f1 score(y val, m.predict proba(X val)[:,1] >= t) for t in ths])]
    pd_ = (pr_ >= th).astype(int)
    rows.append([name, accuracy score(y test,pd ), precision score(y test,pd ),
recall_score(y_test,pd_), f1_score(y_test,pd_), roc_auc_score(y_test,pr_)])
rows.append(["MLP (ours)", accuracy score(y test,pred test), precision score(y test,pred test),
recall_score(y_test,pred_test), f1_score(y_test,pred_test), auc])
bench = pd.DataFrame(rows, columns=["Model","Accuracy","Precision","Recall","F1","ROC-
AUC"]).set_index("Model").round(3)
display(bench)
bench[["F1","ROC-AUC"]].plot.bar(figsize=(8,4), rot=15, color=["#8da0cb","#fc8d62"]); plt.ylim(0.4,
0.9)
```

```
plt.title("Model comparison on the test set"); plt.ylabel("Score")
savefig("r4_model_comparison")
```

**Cell 35**

```
# Permutation importance: shuffle one feature at a time on the validation set and measure the AUC
drop
rng = np.random.default_rng(SEED); base = roc_auc_score(y_val, p_val); imp = {}
for col in X val.columns:
    drops = []
    for   in range(10):
        Xp = X val.copy(); Xp[col] = rng.permutation(Xp[col].values)
        drops.append(base - roc_auc_score(y_val, model.predict(Xp.values, verbose=0).ravel()))
    imp[col] = (np.mean(drops), np.std(drops))
imp = pd.DataFrame(imp, index=["mean","std"]).T.sort_values("mean")
top = imp.tail(15)
plt.figure(figsize=(8, 6)); plt.barh(top.index, top["mean"], xerr=top["std"], color="#66a3d2")
plt.xlabel("Drop in validation AUC when feature is shuffled"); plt.title("Top-15 permutation feature
importances (MLP)")
savefig("r5 permutation importance")
print(imp.sort_values("mean", ascending=False).head(8).round(4).to_string())
```